
UNIT – 1: INTRODUCTION TO PYTHON

1.0 Introduction

Definition

Python is a **high-level, interpreted, general-purpose programming language**. It is easy to learn because of its simple syntax. Python supports object-oriented, procedural, and functional programming.

Python is widely used in web development, artificial intelligence, machine learning, data science, automation, networking, and software development.

Key Points

- High-level programming language.
- Interpreted language (no compilation required).
- Open-source and free to use.
- Easy to read and write.
- Platform-independent.
- Supports object-oriented programming.
- Large collection of built-in libraries.

Advantages

- Simple syntax.
- Easy to learn.
- Faster program development.
- Large community support.
- Cross-platform compatibility.

Disadvantages

- Slower than C and C++.
 - Uses more memory.
 - Not suitable for mobile application development.
 - Less efficient for very high-performance applications.
-

1.1 History of Python

Introduction

Python was developed by **Guido van Rossum** at **Centrum Wiskunde & Informatica (CWI)** in the Netherlands.

Development started in **December 1989**.

The first public version, **Python 0.9.0**, was released in **1991**.

Python was named after the comedy television show "**Monty Python's Flying Circus**", not after the snake.

Today, Python is maintained by the **Python Software Foundation (PSF)**.

Timeline

- **1989** – Development started.
- **1991** – Python 0.9.0 released.
- **2000** – Python 2.0 released.
- **2008** – Python 3.0 released.
- **Present** – Python 3.x is widely used.

Important Features Introduced

- Exception handling.
- Classes and objects.
- Modules.
- Automatic memory management.
- Standard library.

1.2 List Python Features

Python provides many useful features that make programming simple and efficient.

Features of Python

1. Simple and Easy

Python syntax is similar to English, making programs easy to write and understand.

2. Interpreted Language

Python executes code line by line without compilation.

3. High-Level Language

Users focus on problem-solving instead of hardware details.

4. Object-Oriented

Supports classes, objects, inheritance, and polymorphism.

5. Portable

Python programs run on Windows, Linux, and macOS without modification.

6. Free and Open Source

Anyone can download, use, and modify Python.

7. Dynamically Typed

Variable types are assigned automatically during execution.

8. Large Standard Library

Provides modules for networking, mathematics, GUI, file handling, databases, etc.

9. Extensible

Python can be integrated with C and C++ programs.

10. Scalable

Suitable for both small and large applications.

Advantages

- Easy debugging.
 - Less coding.
 - Faster development.
 - Huge community support.
-

1.3 Applications of Python

Python is used in many fields because of its simplicity and powerful libraries.

Applications

1. Web Development

Used to develop websites and web applications using Django and Flask.

2. Artificial Intelligence (AI)

Used to build intelligent systems.

3. Machine Learning

Used for prediction and data analysis.

4. Data Science

Used for data visualization and analytics.

5. Automation

Used to automate repetitive tasks.

6. Game Development

Used to develop games using Pygame.

7. Desktop GUI Applications

Used to create desktop software.

8. Scientific Computing

Used in engineering and scientific research.

9. Cyber Security

Used for ethical hacking, penetration testing, and security automation.

10. Internet of Things (IoT)

Used to control sensors and embedded devices like Raspberry Pi.

Popular Companies Using Python

- Google
- YouTube
- Netflix
- Instagram
- Spotify
- Dropbox
- NASA

1.4 Python Integrated Development and Learning Environment (IDLE)

Definition

IDLE stands for **Integrated Development and Learning Environment**. It is the official Python development environment used to write, edit, run, and debug Python programs.

It is included automatically when Python is installed.

Features of IDLE

1. Python Shell
2. Code Editor
3. Syntax Highlighting
4. Auto Indentation
5. Auto Completion
6. Debugging Support
7. Search and Replace
8. Run Python Programs
9. Interactive Mode
10. Easy User Interface

Components of IDLE

Python Shell

Used to execute Python commands one by one.

Editor Window

Used to write and save Python programs.

Menu Bar

Contains File, Edit, Format, Run, Options, Window, and Help menus.

Debugger

Helps find and correct errors.

Advantages of IDLE

- Free and easy to use.
- Beginner-friendly.
- Built-in debugger.
- Supports syntax highlighting.
- Quick program execution.

Disadvantages

- Limited advanced features.
- Not suitable for very large projects.
- Fewer plugins than professional IDEs.

UNIT – 1 (Continued)

1.5 Explain the Process of Running Python Scripts

Definition

A **Python script** is a file containing Python code with the extension **.py**. It can be executed using the Python interpreter.

Process of Running a Python Script

Method 1: Using Python IDLE

Step 1: Open **Python IDLE**.

Step 2: Click **File** → **New File**.

Step 3: Type the Python program.

Example:

```
print("Hello, World!")
```

Step 4: Save the file with the extension **.py**.

Example:

```
hello.py
```

Step 5: Click **Run** → **Run Module** or press **F5**.

Step 6: The output appears in the Python Shell.

Output

Hello, World!

Method 2: Using Command Prompt

Step 1: Open Command Prompt.

Step 2: Go to the folder where the Python file is saved.

Example:

```
cd D:\Python
```

Step 3: Run the script using:

```
python filename.py
```

Example:

```
python hello.py
```

Output

Hello, World!

Advantages

- Easy to execute.
 - Programs can be modified and rerun quickly.
 - Suitable for beginners.
-
-

1.6 Explain Identifiers, Keywords, Indentation, Variables

A) Identifiers

Definition

Identifiers are the names given to variables, functions, classes, and objects.

Rules for Identifiers

- Must begin with a letter or underscore (_).
- Cannot start with a number.
- Cannot contain special symbols except underscore.
- Keywords cannot be used as identifiers.
- Python is case-sensitive.

Examples

Valid

```
student  
_marks  
age1
```

Invalid

```
1age  
class  
student-name
```

B) Keywords

Definition

Keywords are reserved words that have predefined meanings in Python.

Common Python Keywords

- if
- else

- elif
- for
- while
- break
- continue
- pass
- def
- return
- class
- import
- try
- except
- True
- False
- None

These keywords cannot be used as variable names.

C) Indentation

Definition

Indentation means leaving spaces at the beginning of a line to define a block of code.

Python uses indentation instead of braces { }.

Example

```
age = 18

if age >= 18:
    print("Eligible to Vote")
```

Without proper indentation, Python generates an **IndentationError**.

Advantages

- Makes programs readable.
 - Clearly defines code blocks.
 - Reduces programming mistakes.
-

D) Variables

Definition

A variable is a named memory location used to store data.

Variables do not require explicit declaration in Python.

Example

```
name = "Ravi"  
age = 20  
marks = 90
```

Characteristics

- Stores data.
 - Value can change during program execution.
 - Created automatically when assigned a value.
-
-

1.7 List and Explain Various Data Types

Definition

Data types specify the type of value stored in a variable.

Types of Data in Python

1. Integer (int)

Stores whole numbers.

Example:

```
age = 20
```

2. Float

Stores decimal numbers.

Example:

```
price = 99.99
```

3. Complex

Stores complex numbers.

Example:

```
z = 4 + 5j
```

4. String (str)

Stores text.

Example:

```
name = "Python"
```

5. Boolean (bool)

Stores logical values.

Example:

```
result = True
```

Possible values:

- True
 - False
-

6. List

Stores multiple values.

Example:

```
numbers = [10,20,30]
```

7. Tuple

Stores ordered data that cannot be changed.

Example:

```
marks = (80,90,100)
```

8. Set

Stores unique values.

Example:

```
colors = {"Red", "Blue", "Green"}
```

9. Dictionary

Stores data as key-value pairs.

Example:

```
student = {  
    "Name": "Ravi",  
    "Age": 20  
}
```

Summary Table

Data Type Example

int	10
float	5.5
complex	2+3j
str	"Hello"

Data Type Example

bool	True
list	[1,2,3]
tuple	(1,2,3)
set	{1,2,3}
dictionary	{"a":1}

1.8 Explain Declaration and Initialization of Variables

Declaration

Python does **not require explicit declaration** of variables.

A variable is automatically created when a value is assigned.

Example

```
age = 20
```

Initialization

Initialization means assigning a value to a variable.

Example

```
name = "Ravi"  
marks = 90  
salary = 25000.50
```

Multiple Variable Initialization

```
a = b = c = 100
```

Multiple Variable Assignment

```
x, y, z = 10, 20, 30
```

Reassigning Variables

```
x = 10  
x = 25
```

Output:

```
25
```

Advantages

- Easy variable creation.
 - Less coding.
 - No separate declaration statement.
-
-

1.9 Explain Input and Output Statements

Input Statement

Python uses the **input()** function to receive data from the user.

Syntax

```
variable = input("Message")
```

Example

```
name = input("Enter Name: ")  
print(name)
```

Output

```
Enter Name: Ravi
```

Ravi

Taking Integer Input

```
age = int(input("Enter Age: "))  
print(age)
```

Taking Float Input

```
salary = float(input("Enter Salary: "))  
print(salary)
```

Output Statement

Python uses the **print()** function to display information.

Syntax

```
print(value)
```

Example 1

```
print("Welcome")
```

Output

Welcome

Example 2

```
name = "Ravi"  
age = 20
```

```
print(name)  
print(age)
```

Output

Ravi
20

Printing Multiple Values

```
name = "Ravi"
```

```
age = 20  
  
print(name, age)
```

Output

```
Ravi 20
```

Advantages

- Easy interaction with users.
 - Displays program results clearly.
 - Supports different data types.
-

1.10 Explain Formatted Input and Output

Definition

Formatted input and output is the process of displaying or accepting data in a specific and readable format. Python provides different methods to format the output, making programs easy to understand.

Methods of Formatted Output

1. Using Comma (,)

The `print()` function can display multiple values separated by commas.

Syntax

```
print(value1, value2, value3)
```

Example

```
name = "Ravi"  
age = 20  
  
print("Name:", name)  
print("Age:", age)
```

Output

```
Name: Ravi  
Age: 20
```

2. Using format() Method

The `format()` method inserts values into placeholders `{}`.

Syntax

```
print("{}".format(value))
```

Example

```
name = "Ravi"  
marks = 95  
  
print("Name: {} Marks: {}".format(name, marks))
```

Output

```
Name: Ravi Marks: 95
```

3. Using f-Strings

An f-string is a simple and modern way to format output.

Syntax

```
print(f"Text {variable}")
```

Example

```
name = "Ravi"  
age = 20  
  
print(f"Name: {name}")  
print(f"Age: {age}")
```

Output

```
Name: Ravi  
Age: 20
```

Advantages

- Produces neat and readable output.
 - Easy to display multiple variables.
 - Reduces coding effort.
 - Supports different data types.
-

1.11 State the Usage of Comments

Definition

Comments are statements that are ignored by the Python interpreter. They are used to explain the program and improve readability.

Types of Comments

1. Single-Line Comment

A single-line comment begins with the # symbol.

Syntax

```
# Comment
```

Example

```
# Display a message  
print("Hello")
```

2. Multi-Line Comment

Python uses triple quotes (''' ''' or """ """) for multi-line comments or documentation.

Example

```
"""  
This is  
a multi-line  
comment.  
"""
```

```
print("Python")
```

Uses of Comments

- Explains the program.
 - Improves readability.
 - Makes debugging easier.
 - Helps other programmers understand the code.
 - Temporarily disables code during testing.
-

Advantages

- Easy to understand programs.
 - Better program documentation.
 - Simplifies maintenance.
-

1.12 List and Explain Various Operators

Definition

Operators are symbols used to perform operations on variables and values.

Types of Operators

1. Arithmetic Operators

Used to perform mathematical calculations.

Operator	Description	Example
----------	-------------	---------

+	Addition	a + b
---	----------	-------

-	Subtraction	a - b
---	-------------	-------

*	Multiplication	a * b
---	----------------	-------

Operator Description Example

/	Division	a / b
%	Modulus	a % b
//	Floor Division	a // b
**	Exponent	a ** b

Example

```
a = 10
b = 3

print(a+b)
print(a-b)
print(a*b)
print(a/b)
print(a%b)
print(a//b)
print(a**b)
```

Output

```
13
7
30
3.3333333333333335
1
3
1000
```

2. Comparison (Relational) Operators

Used to compare two values.

Operator	Meaning
==	Equal to
!=	Not Equal to
>	Greater Than
<	Less Than

Operator	Meaning
>=	Greater Than or Equal to
<=	Less Than or Equal to

Example

```
a = 10
b = 5

print(a > b)
print(a == b)
```

Output

```
True
False
```

3. Assignment Operators

Used to assign values.

Operator Example

=	a = 10
+=	a += 5
-=	a -= 5
*=	a *= 2
/=	a /= 2
%=	a %= 2
//=	a //= 2
**=	a **= 2

4. Logical Operators

Used to combine conditions.

Operator	Description
and	Returns True if both conditions are True
or	Returns True if at least one condition is True
not	Reverses the result

Example

```
a = 10  
  
print(a > 5 and a < 20)  
print(not(a > 20))
```

Output

```
True  
True
```

5. Bitwise Operators

Operate on binary numbers.

Operator	Meaning
&	AND
	OR
^	XOR
~	NOT
<<	Left Shift
>>	Right Shift

6. Membership Operators

Used to check whether a value exists in a sequence.

Operator	Meaning
----------	---------

in	Present
----	---------

not in	Not Present
--------	-------------

Example

```
list1 = [10,20,30]
print(20 in list1)
```

Output

True

7. Identity Operators

Compare whether two variables refer to the same object.

Operator	Meaning
----------	---------

is	Same object
----	-------------

is not	Different object
--------	------------------

Example

```
a = 10
b = 10
print(a is b)
```

Output

True

Advantages

- Simplifies calculations.

- Performs comparisons efficiently.
 - Reduces coding effort.
 - Improves program readability.
-

1.13 Explain Boolean Values

Definition

Boolean values represent logical values. Python has only two Boolean values:

- **True**
- **False**

The Boolean data type is represented by `bool`.

Examples

Example 1

```
a = True
b = False

print(a)
print(b)
```

Output

```
True
False
```

Example 2

```
print(10 > 5)
print(10 < 5)
```

Output

```
True
False
```

Uses of Boolean Values

- Decision making.
 - Conditional statements.
 - Loop control.
 - Comparing values.
 - Logical operations.
-

Advantages

- Simplifies decision-making.
 - Makes programs efficient.
 - Used in conditional statements and loops.
-

1.14 Explain Operator Precedence Rules

Definition

Operator precedence determines the order in which operators are evaluated in an expression.

Operators with higher precedence are evaluated before operators with lower precedence.

Order of Precedence

Precedence	Operators
------------	-----------

Highest	() Parentheses
---------	----------------

	** Exponent
--	-------------

	+x, -x (Unary)
--	----------------

	*, /, //, %
--	-------------

	+, -
--	------

Precedence Operators

<, <=, >, >=

==, !=

not

and

Lowest or

Example 1

```
a = 10 + 5 * 2
```

```
print(a)
```

Output

20

Explanation:

- $5 \times 2 = 10$
 - $10 + 10 = 20$
-

Example 2

```
a = (10 + 5) * 2
```

```
print(a)
```

Output

30

Explanation:

- Parentheses are evaluated first.
 - $10 + 5 = 15$
 - $15 \times 2 = 30$
-

Advantages

- Produces correct results.
- Reduces ambiguity in expressions.
- Makes complex calculations easier to understand.

1.15 State the Purpose of Modules

Definition

A **module** is a file that contains Python code such as functions, variables, and classes. It allows programmers to organize code into separate files and reuse it in different programs.

Python modules have the `.py` extension.

Purpose of Modules

- Reuse code in multiple programs.
- Organize programs into separate files.
- Reduce duplicate code.
- Make programs easier to maintain.
- Improve readability.
- Simplify debugging and testing.

Types of Modules

1. Built-in Modules

These are modules provided by Python.

Examples:

- `math`
 - `random`
 - `datetime`
 - `os`
-

2. User-defined Modules

These are modules created by the programmer.

Importing a Module

Syntax

```
import module_name
```

Example

```
import math

print(math.sqrt(25))
```

Output

```
5.0
```

Advantages

- Promotes code reuse.
 - Makes programs modular.
 - Easy to maintain.
 - Saves development time.
-

1.16 Define Functions

Definition

A **function** is a block of reusable code that performs a specific task. Functions help avoid writing the same code repeatedly.

Syntax

```
def function_name():
    statements
```

Example

```
def welcome():  
    print("Welcome to Python")  
  
welcome()
```

Output

```
Welcome to Python
```

Advantages

- Reusability.
 - Reduces code duplication.
 - Makes programs easier to understand.
 - Simplifies debugging.
 - Improves program organization.
-

1.17 List Types of Functions

Python functions are mainly classified into two types.

1. Built-in Functions

These functions are already provided by Python.

Examples

- `print()`
 - `input()`
 - `len()`
 - `type()`
 - `max()`
 - `min()`
-

2. User-defined Functions

These are functions created by the programmer using the `def` keyword.

Example

```
def add():  
    print(10 + 20)  
  
add()
```

Output

30

Advantages

- Easy to reuse.
 - Reduces program length.
 - Improves readability.
-

1.18 List and Explain Built-in Functions

Definition

Built-in functions are predefined functions available in Python. They perform common tasks without requiring the programmer to define them.

Common Built-in Functions

1. `print()`

Displays output on the screen.

Example

```
print("Hello")
```

Output

Hello

2. input()

Accepts input from the user.

Example

```
name = input("Enter Name: ")  
print(name)
```

3. type()

Returns the data type of a variable.

Example

```
x = 10  
print(type(x))
```

Output

```
<class 'int'>
```

4. len()

Returns the number of elements in a string, list, or tuple.

Example

```
name = "Python"  
print(len(name))
```

Output

```
6
```

5. max()

Returns the largest value.

Example

```
print(max(10, 20, 30))
```

Output

```
30
```

6. min()

Returns the smallest value.

Example

```
print(min(10, 20, 30))
```

Output

```
10
```

7. sum()

Returns the sum of all elements.

Example

```
numbers = [10, 20, 30]  
print(sum(numbers))
```

Output

```
60
```

8. abs()

Returns the absolute (positive) value.

Example

```
print(abs(-25))
```

Output

```
25
```

9. round()

Rounds a number to the nearest integer or specified decimal places.

Example

```
print(round(5.67))
```

Output

6

10. sorted()

Returns the elements in sorted order.

Example

```
numbers = [40, 10, 30, 20]  
print(sorted(numbers))
```

Output

[10, 20, 30, 40]

Advantages

- Saves programming time.
 - Easy to use.
 - Improves code readability.
 - Reduces program length.
-

1.19 Explain the Steps in Developing a Simple Python Program and Execution

Definition

Developing a Python program involves writing, testing, and executing the program to obtain the required output.

Steps in Developing a Python Program

Step 1: Identify the Problem

Understand the problem and determine the required output.

Step 2: Design the Solution

Prepare an algorithm or flowchart to solve the problem.

Step 3: Write the Python Program

Write the program using Python syntax.

Example

```
print("Hello, Python!")
```

Step 4: Save the Program

Save the file with the **.py** extension.

Example

```
hello.py
```

Step 5: Execute the Program

Using IDLE

1. Open Python IDLE.
2. Click **File** → **New File**.
3. Write the program.
4. Save the file.
5. Press **F5** or select **Run** → **Run Module**.

Using Command Prompt

1. Open Command Prompt.
2. Go to the folder containing the Python file.
3. Execute the program using:

```
python hello.py
```

Step 6: Verify the Output

Check whether the output is correct. If there are errors, correct them and execute the program again.

Example Program

```
name = input("Enter your name: ")  
print("Welcome", name)
```

Sample Output

```
Enter your name: Ravi  
Welcome Ravi
```

Advantages

- Helps solve problems systematically.
- Makes programs easier to develop and test.
- Reduces errors through proper testing.
- Produces reliable and accurate results.

UNIT – 2 : CONTROL FLOW AND LOOPS

2.0 Control Flow and Loops

Definition

Control flow is the order in which program statements are executed. It allows the program to make decisions and repeat tasks based on conditions.

Loops are used to execute a block of code repeatedly until a specified condition becomes false.

Importance of Control Flow and Loops

- Makes programs intelligent by taking decisions.
 - Reduces repetition of code.
 - Saves programming time.
 - Improves program efficiency.
 - Simplifies complex problems.
-

2.1 List and Explain Various Control Flow Constructs

Control flow constructs are used to make decisions in a program based on conditions.

Python provides the following control flow statements:

- if
 - if-else
 - if-elif-else
-

2.1.1 if Statement

Definition

The **if** statement is used to execute a block of code only when the given condition is **True**.

Syntax

```
if condition:  
    statements
```

Flow of Execution

1. Check the condition.
 2. If the condition is **True**, execute the statements.
 3. If the condition is **False**, skip the statements.
-

Example

```
age = 20  
  
if age >= 18:  
    print("Eligible to Vote")
```

Output

```
Eligible to Vote
```

Advantages

- Executes code only when required.
 - Simple and easy to understand.
 - Improves decision making.
-

2.1.2 if-else Statement

Definition

The **if-else** statement is used when two different actions are required based on a condition.

Syntax

```
if condition:
    statements
else:
    statements
```

Working

- If the condition is **True**, the **if** block executes.
 - Otherwise, the **else** block executes.
-

Example

```
number = 5

if number % 2 == 0:
    print("Even Number")
else:
    print("Odd Number")
```

Output

Odd Number

Advantages

- Provides two possible outputs.
 - Simplifies decision making.
 - Easy to read.
-

2.1.3 if-elif-else Statement

Definition

The **if-elif-else** statement is used to check multiple conditions.

Syntax

```
if condition1:
    statements
elif condition2:
    statements
else:
    statements
```

Working

- The conditions are checked one by one.
 - The first condition that becomes **True** is executed.
 - If no condition is true, the **else** block is executed.
-

Example

```
marks = 75

if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 50:
    print("Grade C")
else:
    print("Fail")
```

Output

Grade B

Advantages

- Handles multiple conditions.

- Reduces nested if statements.
 - Makes programs easier to understand.
-

2.2 List and Explain Various Loop Statements

Definition

Loop statements execute a block of code repeatedly until a condition becomes false.

Python provides two looping statements:

- for loop
- while loop

Loop control statements:

- break
 - continue
 - pass
-

2.2.1 for Loop

Definition

The **for** loop is used to execute a block of code for each item in a sequence such as a list, tuple, string, or range.

Syntax

```
for variable in sequence:  
    statements
```

Example 1

```
for i in range(1,6):  
    print(i)
```

Output

```
1  
2  
3  
4  
5
```

Example 2

```
name = "PYTHON"  
  
for letter in name:  
    print(letter)
```

Output

```
P  
Y  
T  
H  
O  
N
```

Advantages

- Easy to use.
 - Suitable for sequences.
 - Reduces code repetition.
-

2.2.2 while Loop

Definition

The **while** loop executes a block of code repeatedly as long as the condition remains **True**.

Syntax

```
while condition:  
    statements
```

Example

```
i = 1

while i <= 5:
    print(i)
    i += 1
```

Output

```
1
2
3
4
5
```

Advantages

- Suitable when the number of iterations is unknown.
 - Flexible and easy to understand.
 - Useful for condition-based repetition.
-

2.2.3 break Statement

Definition

The **break** statement immediately terminates the loop when a specified condition is satisfied.

Syntax

```
break
```

Example

```
for i in range(1,10):
    if i == 6:
        break
    print(i)
```

Output

1
2
3
4
5

Advantages

- Stops unnecessary loop execution.
 - Improves program efficiency.
 - Saves execution time.
-

2.2.4 continue Statement

Definition

The **continue** statement skips the current iteration and executes the next iteration of the loop.

Syntax

```
continue
```

Example

```
for i in range(1,6):  
    if i == 3:  
        continue  
    print(i)
```

Output

1
2
4
5

Advantages

- Skips unwanted iterations.

- Makes programs simple.
 - Improves readability.
-

2.2.5 pass Statement

Definition

The **pass** statement is a null statement. It does nothing when executed and is used as a placeholder for future code.

Syntax

```
pass
```

Example

```
for i in range(1,6):  
    if i == 3:  
        pass  
    print(i)
```

Output

```
1  
2  
3  
4  
5
```

Uses of `pass`

- Acts as a placeholder.
 - Used while developing programs.
 - Prevents syntax errors when a block of code is empty.
-

Advantages

- Allows writing incomplete programs.

- Useful during program development.
- Improves code structure.

UNIT – 3 : FUNCTIONS AND ARRAYS

3.0 Functions and Arrays

Definition

Functions are reusable blocks of code that perform a specific task. They help in reducing code repetition and make programs easier to understand and maintain.

Arrays are data structures used to store multiple values of the same data type in a single variable. In Python, arrays can be created using the **array** module.

Importance of Functions

- Reuse code.
 - Reduce program size.
 - Improve readability.
 - Simplify debugging.
 - Make programs modular.
-

Importance of Arrays

- Store multiple values in one variable.
 - Easy access to data using indexes.
 - Efficient memory utilization.
 - Faster data processing.
-

3.1 Introduction

Definition

A function is a named block of code that performs a particular task. Once a function is created, it can be called multiple times from different parts of the program.

Python provides two types of functions:

- Built-in functions
- User-defined functions

Functions improve the modularity and reusability of programs.

Syntax

```
def function_name():  
    statements  
  
function_name()
```

Example

```
def welcome():  
    print("Welcome to Python Programming")  
  
welcome()
```

Output

Welcome to Python Programming

Advantages

- Reduces code duplication.
 - Improves readability.
 - Makes debugging easier.
 - Promotes code reusability.
 - Simplifies program maintenance.
-

3.2 Function Arguments: Default Arguments, Variable Length Arguments

Definition

Arguments are the values passed to a function when it is called. They allow a function to receive data and perform operations on it.

Python supports different types of function arguments, including **Default Arguments** and **Variable Length Arguments**.

A) Default Arguments

Definition

A default argument is an argument that has a predefined value. If no value is passed during the function call, the default value is used automatically.

Syntax

```
def function_name(parameter=value):  
    statements
```

Example

```
def greet(name="Student"):  
    print("Welcome", name)  
  
greet()  
greet("Ravi")
```

Output

```
Welcome Student  
Welcome Ravi
```

Advantages

- Reduces unnecessary arguments.
 - Makes function calls simple.
 - Improves flexibility.
-

B) Variable Length Arguments

Definition

Variable length arguments allow a function to accept any number of arguments. They are represented using the * symbol.

Syntax

```
def function_name(*arguments):  
    statements
```

Example

```
def total(*numbers):  
    print(sum(numbers))  
  
total(10,20,30)  
total(5,10,15,20)
```

Output

```
60  
50
```

Advantages

- Accepts multiple values.
 - Makes functions flexible.
 - Reduces the need to define many parameters.
-

3.3 Anonymous Functions (Lambda Functions)

Definition

An anonymous function is a function without a name. It is created using the **lambda** keyword and is mainly used for short and simple operations.

Syntax

```
lambda arguments: expression
```

Example 1

```
square = lambda x: x*x  
  
print(square(5))
```

Output

```
25
```

Example 2

```
add = lambda a,b: a+b  
  
print(add(10,20))
```

Output

```
30
```

Advantages

- Requires less code.
 - Easy to write.
 - Suitable for small functions.
 - Improves readability for simple operations.
-

Disadvantages

- Can contain only one expression.
 - Not suitable for complex programs.
 - Difficult to debug.
-

3.4 Return Statement

Definition

The `return` statement is used to send a value from a function back to the function call. After executing the `return` statement, the function terminates.

Syntax

```
def function_name():  
    return value
```

Example

```
def add(a,b):  
    return a+b  
  
result = add(10,20)  
print(result)
```

Output

30

Example 2

```
def square(num):  
    return num*num  
  
print(square(6))
```

Output

36

Advantages

- Returns values to the calling function.
 - Improves code reusability.
 - Makes functions more useful.
-

3.5 List and Explain Scope of Variables

Definition

The scope of a variable refers to the region of a program where the variable can be accessed.

Python mainly has two types of variable scope.

1. Local Variable

Definition

A local variable is declared inside a function and can be accessed only within that function.

Example

```
def display():  
    x = 100  
    print(x)
```

```
display()
```

Output

```
100
```

The variable `x` cannot be accessed outside the function.

2. Global Variable

Definition

A global variable is declared outside all functions and can be accessed throughout the program.

Example

```
x = 50

def show():
    print(x)

show()
print(x)
```

Output

```
50
50
```

Difference Between Local and Global Variables

Local Variable	Global Variable
Declared inside a function	Declared outside a function
Accessible only within the function	Accessible throughout the program
Exists only during function execution	Exists until the program ends

Advantages of Variable Scope

- Prevents naming conflicts.
- Improves program security.
- Makes debugging easier.
- Organizes the program efficiently.
- Supports modular programming.

3.6 Explain Creation of Modules

Definition

A **module** is a Python file (.py) that contains functions, variables, classes, and statements. Modules help organize code into separate files and allow the same code to be reused in multiple programs.

Steps to Create a Module

Step 1: Open a Python editor (IDLE or any IDE).

Step 2: Write the required functions or variables.

Example (mymodule.py)

```
def add(a, b):  
    return a + b  
  
def sub(a, b):  
    return a - b
```

Step 3: Save the file with the .py extension.

Example:

mymodule.py

Step 4: Import the module into another Python program and use its functions.

Advantages

- Code reusability.
 - Reduces duplicate code.
 - Easy to maintain.
 - Improves readability.
 - Supports modular programming.
-

3.7 Explain Importing of Modules

Definition

Importing a module means using the functions, variables, or classes defined in another Python file.

Python provides different ways to import modules.

1. Using `import` Statement

Syntax

```
import module_name
```

Example

```
import math

print(math.sqrt(25))
```

Output

```
5.0
```

2. Using `from...import` Statement

Syntax

```
from module_name import function_name
```

Example

```
from math import sqrt

print(sqrt(49))
```

Output

```
7.0
```

3. Using `import ... as` Statement

This method gives an alias (short name) to a module.

Syntax

```
import module_name as alias_name
```

Example

```
import math as m

print(m.factorial(5))
```

Output

```
120
```

Advantages

- Reuses existing code.
 - Saves programming time.
 - Improves program organization.
 - Makes programs modular.
-

3.8 Python Variable: Namespace and Scoping

Definition

A **namespace** is a collection of names (identifiers) associated with objects. It helps Python organize variables, functions, and classes and prevents name conflicts.

Scoping determines where a variable can be accessed in a program.

Types of Namespaces

1. Built-in Namespace

Contains predefined functions and keywords provided by Python.

Examples

- `print()`
- `len()`

- `input()`
-

2. Global Namespace

Contains variables created outside all functions.

Example

```
x = 100

def display():
    print(x)

display()
```

Output

```
100
```

3. Local Namespace

Contains variables created inside a function.

Example

```
def show():
    y = 50
    print(y)

show()
```

Output

```
50
```

Variable Scoping

Python searches for variables in the following order (LEGB Rule):

1. **Local Scope (L)**
2. **Enclosing Scope (E)**
3. **Global Scope (G)**
4. **Built-in Scope (B)**

Advantages

- Prevents naming conflicts.
 - Organizes variables efficiently.
 - Improves program readability.
 - Supports modular programming.
-

3.9 Explain Python Packages

Definition

A **package** is a collection of related Python modules stored in a directory. Packages help organize large Python projects.

A package usually contains an `__init__.py` file.

Structure of a Package

```
MyPackage/  
├── __init__.py  
├── add.py  
├── sub.py  
└── multiply.py
```

Importing a Package

Syntax

```
import package_name.module_name
```

Example

```
import math  
  
print(math.pi)
```

Output

```
3.141592653589793
```

Advantages

- Organizes modules efficiently.
 - Easy to maintain large projects.
 - Supports code reuse.
 - Avoids naming conflicts.
-

3.10 List and Explain Strings: String Slices, Immutability

Definition

A **string** is a sequence of characters enclosed in single quotes (' '), double quotes (" "), or triple quotes (''' ''' or """ """).

Strings are one of the most commonly used data types in Python.

Example

```
name = "Python"  
print(name)
```

Output

```
Python
```

String Slicing

Definition

String slicing is the process of extracting a portion of a string using indexes.

Syntax

```
string[start : stop : step]
```

Where:

- **start** → Starting index.
 - **stop** → Ending index (not included).
 - **step** → Number of positions to skip.
-

Example 1

```
text = "PYTHON"  
  
print(text[1:5])
```

Output

```
YTHO
```

Example 2

```
text = "PYTHON"  
  
print(text[:4])
```

Output

```
PYTH
```

Example 3

```
text = "PYTHON"  
  
print(text[-3:])
```

Output

```
HON
```

String Immutability

Definition

Strings in Python are **immutable**, which means their contents cannot be changed after they are created. If a modification is needed, a new string must be created.

Example

```
name = "Python"

name = "Java"

print(name)
```

Output

Java

In the above example, the original string is not modified. Instead, a new string is created and assigned to the variable.

Advantages of Strings

- Easy to store and manipulate text.
- Supports indexing and slicing.
- Immutable, making programs more secure and reliable.
- Provides many built-in methods for text processing.

3.11 List and Explain String Functions and Methods

Definition

String functions and methods are built-in functions used to perform operations such as converting case, searching, replacing, splitting, and joining strings.

Common String Functions and Methods

1. len()

Returns the length of a string.

Example

```
text = "Python"  
print(len(text))
```

Output

6

2. upper()

Converts all characters to uppercase.

Example

```
text = "python"  
print(text.upper())
```

Output

PYTHON

3. lower()

Converts all characters to lowercase.

Example

```
text = "PYTHON"  
print(text.lower())
```

Output

python

4. capitalize()

Converts the first character into uppercase.

Example

```
text = "python"  
print(text.capitalize())
```

Output

Python

5. title()

Converts the first letter of every word into uppercase.

Example

```
text = "python programming"  
print(text.title())
```

Output

Python Programming

6. strip()

Removes spaces from both ends of a string.

Example

```
text = " Python "  
print(text.strip())
```

Output

Python

7. replace()

Replaces one string with another.

Syntax

```
string.replace(old, new)
```

Example

```
text = "Hello World"  
print(text.replace("World", "Python"))
```

Output

```
Hello Python
```

8. split()

Splits a string into a list.

Example

```
text = "Python is Easy"  
print(text.split())
```

Output

```
['Python', 'is', 'Easy']
```

9. join()

Joins elements into a single string.

Example

```
words = ["Python", "Programming"]  
print(" ".join(words))
```

Output

```
Python Programming
```

10. find()

Returns the position of the first occurrence of a character or substring.

Example

```
text = "Python"  
print(text.find("t"))
```

Output

```
2
```

11. count()

Returns the number of occurrences of a character or substring.

Example

```
text = "banana"
print(text.count("a"))
```

Output

3

12. startswith()

Checks whether the string starts with the specified value.

Example

```
text = "Python"
print(text.startswith("Py"))
```

Output

True

13. endswith()

Checks whether the string ends with the specified value.

Example

```
text = "Python"
print(text.endswith("on"))
```

Output

True

Advantages

- Easy manipulation of strings.
- Reduces coding effort.
- Improves readability.

- Provides many built-in operations.
-

3.12 Explain About String Module

Definition

The **string** module is a built-in Python module that provides useful constants and helper functions for string processing.

To use it, the module must first be imported.

Syntax

```
import string
```

Common Constants in the String Module

1. ascii_letters

Contains all uppercase and lowercase English letters.

Example

```
import string
print(string.ascii_letters)
```

2. ascii_lowercase

Contains lowercase letters (a-z).

Example

```
import string
print(string.ascii_lowercase)
```

3. ascii_uppercase

Contains uppercase letters (A-Z).

Example

```
import string
print(string.ascii_uppercase)
```

4. digits

Contains digits from 0 to 9.

Example

```
import string
print(string.digits)
```

Output

```
0123456789
```

5. punctuation

Contains punctuation symbols.

Example

```
import string
print(string.punctuation)
```

6. whitespace

Contains space, tab, and newline characters.

Example

```
import string
print(string.whitespace)
```

Advantages

- Provides predefined constants.
 - Simplifies string processing.
 - Saves programming time.
 - Useful in text manipulation.
-

3.13 Explain About Python Arrays

Definition

An **array** is a collection of elements of the same data type stored in contiguous memory locations. In Python, arrays are created using the **array** module.

Importing the Array Module

```
from array import *
```

Creating an Array

Syntax

```
array(typecode, [elements])
```

Example

```
from array import *  
  
numbers = array('i', [10, 20, 30, 40])  
print(numbers)
```

Output

```
array('i', [10, 20, 30, 40])
```

Common Type Codes

Type Code	Data Type
i	Integer
f	Float
d	Double
u	Unicode Character

Advantages

- Stores multiple values efficiently.
 - Requires less memory.
 - Fast access using indexes.
 - Suitable for storing similar data.
-

3.14 Explain Accessing of Elements in an Array

Definition

Array elements are accessed using **index numbers**. The first element has index **0**.

Syntax

```
array_name[index]
```

Example

```
from array import *  
  
numbers = array('i', [10, 20, 30, 40])  
  
print(numbers[0])  
print(numbers[2])
```

Output

```
10  
30
```

Accessing Elements Using a Loop

Example

```
from array import *  
  
numbers = array('i', [10, 20, 30, 40])  
  
for i in numbers:  
    print(i)
```

Output

```
10  
20  
30  
40
```

Advantages

- Easy retrieval of elements.
 - Fast access using indexes.
 - Supports iteration using loops.
-

3.15 Explain Array Methods

Definition

Array methods are built-in methods used to perform operations such as adding, removing, searching, and reversing elements in an array.

Common Array Methods

1. append()

Adds an element at the end.

Example

```
from array import *  
  
a = array('i', [10, 20])  
a.append(30)  
  
print(a)
```

Output

```
array('i', [10, 20, 30])
```

2. insert()

Inserts an element at a specified position.

Syntax

```
array.insert(index, value)
```

Example

```
from array import *

a = array('i', [10, 30])
a.insert(1, 20)

print(a)
```

Output

```
array('i', [10, 20, 30])
```

3. remove()

Removes the specified element.

Example

```
from array import *

a = array('i', [10, 20, 30])
a.remove(20)

print(a)
```

Output

```
array('i', [10, 30])
```

4. pop()

Removes and returns the last element.

Example

```
from array import *

a = array('i', [10, 20, 30])
a.pop()
```

```
print(a)
```

Output

```
array('i', [10, 20])
```

5. index()

Returns the index of the specified element.

Example

```
from array import *  
  
a = array('i', [10, 20, 30])  
print(a.index(20))
```

Output

```
1
```

6. reverse()

Reverses the elements of the array.

Example

```
from array import *  
  
a = array('i', [10, 20, 30])  
a.reverse()  
  
print(a)
```

Output

```
array('i', [30, 20, 10])
```

7. extend()

Adds all elements of another array to the existing array.

Example

```
from array import *
```

```
a = array('i', [10, 20])
b = array('i', [30, 40])

a.extend(b)

print(a)
```

Output

```
array('i', [10, 20, 30, 40])
```

Advantages

- Easy insertion and deletion of elements.
- Simplifies array operations.
- Improves program efficiency.
- Provides quick manipulation of array data.

UNIT – 4 : DATA STRUCTURES

4.0 Data Structures

Definition

A **data structure** is a way of organizing and storing data so that it can be accessed and modified efficiently. Python provides several built-in data structures such as **Lists, Tuples, Sets, and Dictionaries**.

Data structures help programmers manage large amounts of data efficiently.

Types of Data Structures in Python

- List
 - Tuple
 - Set
 - Dictionary
-

Advantages

- Organizes data efficiently.
 - Makes searching and processing easier.
 - Reduces program complexity.
 - Improves performance.
 - Simplifies data management.
-

4.1 Explain Python Lists

Definition

A **List** is an ordered collection of elements enclosed within square brackets []. Lists can store different types of data such as integers, floats, strings, and even other lists.

Lists are **mutable**, which means their elements can be changed after creation.

Characteristics of Lists

- Ordered collection.
 - Mutable (modifiable).
 - Allows duplicate elements.
 - Supports indexing and slicing.
 - Can store different data types.
-

Syntax

```
list_name = [element1, element2, element3]
```

Example

```
students = ["Ravi", "Sita", "Rahul"]  
print(students)
```

Output

```
['Ravi', 'Sita', 'Rahul']
```

Advantages

- Easy to create and modify.
 - Stores multiple values.
 - Supports indexing and slicing.
 - Allows duplicate values.
-

4.2 Describe Basic List Operations

Definition

List operations are the operations performed on lists such as accessing, adding, updating, deleting, and concatenating elements.

1. Accessing Elements

Elements are accessed using index numbers.

Example

```
numbers = [10, 20, 30, 40]

print(numbers[0])
print(numbers[2])
```

Output

```
10
30
```

2. Updating Elements

Elements can be modified using their index.

Example

```
numbers = [10, 20, 30]

numbers[1] = 50

print(numbers)
```

Output

```
[10, 50, 30]
```

3. Adding Elements

New elements can be added using `append()`.

Example

```
numbers = [10, 20]

numbers.append(30)

print(numbers)
```

Output

```
[10, 20, 30]
```

4. Deleting Elements

Elements can be deleted using `remove()` or `del`.

Example

```
numbers = [10, 20, 30]
numbers.remove(20)
print(numbers)
```

Output

```
[10, 30]
```

5. Concatenation

Two lists can be joined using the `+` operator.

Example

```
list1 = [1, 2]
list2 = [3, 4]
print(list1 + list2)
```

Output

```
[1, 2, 3, 4]
```

6. Repetition

A list can be repeated using the `*` operator.

Example

```
numbers = [1, 2]
print(numbers * 3)
```

Output

```
[1, 2, 1, 2, 1, 2]
```

Advantages

- Easy manipulation of data.
 - Supports multiple operations.
 - Reduces programming effort.
-

4.3 Explain List Slices

Definition

List slicing is the process of extracting a portion of a list using index values.

Syntax

```
list_name[start : stop : step]
```

Where:

- **start** → Starting index.
 - **stop** → Ending index (not included).
 - **step** → Interval between elements.
-

Example 1

```
numbers = [10, 20, 30, 40, 50]  
print(numbers[1:4])
```

Output

```
[20, 30, 40]
```

Example 2

```
numbers = [10, 20, 30, 40, 50]  
print(numbers[:3])
```

Output

```
[10, 20, 30]
```

Example 3

```
numbers = [10, 20, 30, 40, 50]  
print(numbers[-2:])
```

Output

```
[40, 50]
```

Advantages

- Extracts required elements easily.
 - Simplifies list processing.
 - Improves program readability.
-

4.4 Explain List Methods

Definition

List methods are built-in functions used to perform various operations on lists.

1. append()

Adds an element to the end of the list.

Example

```
numbers = [10, 20]  
numbers.append(30)  
print(numbers)
```

Output

```
[10, 20, 30]
```

2. insert()

Inserts an element at a specified position.

Example

```
numbers = [10, 30]
numbers.insert(1, 20)
print(numbers)
```

Output

```
[10, 20, 30]
```

3. remove()

Removes the specified element.

Example

```
numbers = [10, 20, 30]
numbers.remove(20)
print(numbers)
```

Output

```
[10, 30]
```

4. pop()

Removes and returns the last element.

Example

```
numbers = [10, 20, 30]
numbers.pop()
print(numbers)
```

Output

```
[10, 20]
```

5. sort()

Sorts the list in ascending order.

Example

```
numbers = [30, 10, 20]
numbers.sort()
print(numbers)
```

Output

```
[10, 20, 30]
```

6. reverse()

Reverses the order of elements.

Example

```
numbers = [10, 20, 30]
numbers.reverse()
print(numbers)
```

Output

```
[30, 20, 10]
```

7. count()

Returns the number of occurrences of an element.

Example

```
numbers = [10, 20, 20, 30]
print(numbers.count(20))
```

Output

2

8. index()

Returns the index of the specified element.

Example

```
numbers = [10, 20, 30]
print(numbers.index(20))
```

Output

1

Advantages

- Simplifies list manipulation.
 - Saves programming time.
 - Improves readability.
-

4.5 Explain List Loop

Definition

A **list loop** is used to access and process each element of a list one by one. Python mainly uses the **for loop** to iterate through lists.

Using for Loop

Syntax

```
for variable in list_name:
    statements
```

Example

```
numbers = [10, 20, 30, 40]

for i in numbers:
    print(i)
```

Output

```
10
20
30
40
```

Using while Loop

Example

```
numbers = [10, 20, 30]

i = 0

while i < len(numbers):
    print(numbers[i])
    i += 1
```

Output

```
10
20
30
```

Advantages

- Processes all list elements efficiently.
- Reduces repetitive code.
- Makes programs simple and readable.
- Useful for searching and displaying list elements.

4.6 Explain Mutability

Definition

Mutability means the ability to change or modify the contents of an object after it has been created. In Python, **lists are mutable**, so their elements can be added, removed, or updated without creating a new list.

Example

```
numbers = [10, 20, 30]

numbers[1] = 50

print(numbers)
```

Output

```
[10, 50, 30]
```

Advantages

- Easy to modify data.
 - Supports insertion and deletion of elements.
 - Suitable for dynamic data storage.
 - Saves memory by modifying the existing object.
-

4.7 Explain Aliasing

Definition

Aliasing means assigning one list to another variable. Both variables refer to the **same list object**. Any changes made using one variable will also affect the other.

Syntax

```
list2 = list1
```

Example

```
list1 = [10, 20, 30]

list2 = list1

list2.append(40)

print("List1:", list1)
```

```
print("List2:", list2)
```

Output

```
List1: [10, 20, 30, 40]  
List2: [10, 20, 30, 40]
```

Advantages

- Saves memory.
 - Easy to share the same data between variables.
 - Simple to implement.
-

Disadvantages

- Changes in one list affect the other.
 - May cause unexpected results.
 - Not suitable when independent copies are required.
-

4.8 Explain Cloning Lists

Definition

Cloning means creating a new list that contains the same elements as the original list. Unlike aliasing, changes made to the cloned list do not affect the original list.

Method 1: Using `copy()`

Example

```
list1 = [10, 20, 30]  
  
list2 = list1.copy()  
  
list2.append(40)  
  
print("List1:", list1)  
print("List2:", list2)
```

Output

```
List1: [10, 20, 30]  
List2: [10, 20, 30, 40]
```

Method 2: Using Slicing

Example

```
list1 = [10, 20, 30]  
  
list2 = list1[:]  
  
print(list2)
```

Output

```
[10, 20, 30]
```

Advantages

- Creates an independent copy.
 - Prevents accidental modification of the original list.
 - Easy to use.
-

4.9 Explain List Parameters

Definition

List parameters are lists that are passed as arguments to functions. A function can receive a list, process its elements, and return the result.

Syntax

```
def function_name(list_name):  
    statements
```

Example

```
def display(numbers):
```

```
    for i in numbers:
        print(i)

list1 = [10, 20, 30]

display(list1)
```

Output

```
10
20
30
```

Advantages

- Reduces code duplication.
 - Allows processing of multiple elements.
 - Improves code reusability.
 - Makes programs modular.
-

4.10 Explain List Comprehension

Definition

List comprehension is a simple and concise way to create a new list from an existing sequence using a single line of code.

Syntax

```
[expression for variable in sequence]
```

Example 1

```
numbers = [1, 2, 3, 4, 5]

square = [x*x for x in numbers]

print(square)
```

Output

```
[1, 4, 9, 16, 25]
```

Example 2

```
even = [x for x in range(1,11) if x % 2 == 0]  
print(even)
```

Output

```
[2, 4, 6, 8, 10]
```

Advantages

- Short and easy to write.
- Reduces the number of code lines.
- Improves readability.
- Faster than using loops for simple list creation.
- Makes programs more efficient.

4.11 Tuples

Definition

A **Tuple** is an ordered collection of elements enclosed in parentheses `()`. Tuples can store different types of data such as integers, strings, and floats.

Unlike lists, **tuples are immutable**, which means their elements cannot be changed after creation.

Characteristics of Tuples

- Ordered collection.
 - Immutable (cannot be modified).
 - Allows duplicate elements.
 - Supports indexing and slicing.
 - Can store different data types.
-

Syntax

```
tuple_name = (element1, element2, element3)
```

Example

```
student = ("Ravi", 20, "CSE")  
  
print(student)
```

Output

```
('Ravi', 20, 'CSE')
```

Advantages

- Faster than lists.
 - Protects data from accidental modification.
 - Uses less memory.
 - Suitable for fixed data.
-

4.11.1 Explain Tuple Assignment

Definition

Tuple assignment is the process of assigning multiple values to multiple variables in a single statement using a tuple.

Syntax

```
variable1, variable2 = value1, value2
```

Example

```
name, age = ("Ravi", 20)  
  
print(name)  
print(age)
```

Output

```
Ravi  
20
```

Swapping Variables Using Tuple Assignment

Example

```
a = 10  
b = 20  
  
a, b = b, a  
  
print(a)  
print(b)
```

Output

```
20  
10
```

Advantages

- Simple and easy.
 - Reduces code.
 - No temporary variable required for swapping.
-

4.11.2 Explain Tuple as Return Value

Definition

A Python function can return multiple values by returning them as a tuple.

Syntax

```
def function_name():  
    return value1, value2
```

Example

```
def calculate(a, b):
```

```
        return a+b, a-b

sum1, diff = calculate(20, 10)

print(sum1)
print(diff)
```

Output

```
30
10
```

Advantages

- Returns multiple values together.
 - Makes functions more useful.
 - Improves code readability.
-

4.11.3 Explain Tuple Comprehension

Definition

Python **does not support tuple comprehension directly** like list comprehension. Instead, Python uses **generator expressions**, which can be converted into tuples using the `tuple()` function.

Syntax

```
tuple(expression for variable in sequence)
```

Example

```
numbers = tuple(x*x for x in range(1,6))

print(numbers)
```

Output

```
(1, 4, 9, 16, 25)
```

Advantages

- Creates tuples efficiently.
 - Reduces code.
 - Easy to understand.
-

4.12 Dictionaries

Definition

A **Dictionary** is a collection of **key-value pairs** enclosed in curly braces `{ }`. Each key must be unique and is used to access its corresponding value.

Characteristics

- Stores data as key-value pairs.
 - Mutable.
 - Keys must be unique.
 - Fast data retrieval.
 - Supports different data types.
-

Syntax

```
dictionary_name = {  
    key1: value1,  
    key2: value2  
}
```

Example

```
student = {  
    "Name": "Ravi",  
    "Age": 20,  
    "Course": "CSE"  
}  
  
print(student)
```

Output

```
{'Name': 'Ravi', 'Age': 20, 'Course': 'CSE'}
```

Advantages

- Fast searching.
 - Easy data organization.
 - Stores related information together.
 - Supports dynamic updates.
-

4.12.1 Explain Creation of Dictionary / Assignment

Creating a Dictionary

Example

```
student = {  
    "Name": "Ravi",  
    "Age": 20  
}  
  
print(student)
```

Output

```
{'Name': 'Ravi', 'Age': 20}
```

Assigning or Updating Values

Example

```
student = {  
    "Name": "Ravi"  
}  
  
student["Age"] = 20  
  
print(student)
```

Output

```
{'Name': 'Ravi', 'Age': 20}
```

Advantages

- Easy to create.
 - Supports updating values.
 - Flexible data storage.
-

4.12.2 Explain Operations and Methods

Dictionary Operations

1. Accessing Values

```
student = {  
    "Name": "Ravi",  
    "Age": 20  
}  
  
print(student["Name"])
```

Output

Ravi

2. Adding a New Item

```
student = {  
    "Name": "Ravi"  
}  
  
student["Age"] = 20  
  
print(student)
```

Output

```
{'Name': 'Ravi', 'Age': 20}
```

3. Deleting an Item

```
student = {  
    "Name": "Ravi",  
    "Age": 20  
}  
  
del student["Age"]
```

```
print(student)
```

Output

```
{'Name': 'Ravi'}
```

Common Dictionary Methods

1. keys()

Returns all keys.

```
student = {"Name": "Ravi", "Age": 20}

print(student.keys())
```

2. values()

Returns all values.

```
student = {"Name": "Ravi", "Age": 20}

print(student.values())
```

3. items()

Returns key-value pairs.

```
student = {"Name": "Ravi", "Age": 20}

print(student.items())
```

4. get()

Returns the value of a specified key.

```
student = {"Name": "Ravi"}

print(student.get("Name"))
```

Output

```
Ravi
```

5. pop()

Removes a specified key.

```
student = {  
    "Name": "Ravi",  
    "Age": 20  
}  
  
student.pop("Age")  
  
print(student)
```

Output

```
{'Name': 'Ravi'}
```

Advantages

- Fast operations.
 - Easy retrieval of data.
 - Supports modification.
-

4.12.3 Explain Dictionary Comprehension

Definition

Dictionary comprehension is a concise way to create dictionaries using a single line of code.

Syntax

```
{key:value for variable in sequence}
```

Example

```
square = {x:x*x for x in range(1,6)}  
  
print(square)
```

Output

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Advantages

- Short and simple.
 - Improves readability.
 - Faster than traditional loops.
-

4.13 Explain Sets

Definition

A **Set** is an unordered collection of unique elements enclosed in curly braces `{ }`. Sets automatically remove duplicate values.

Characteristics

- Unordered collection.
 - Mutable.
 - Does not allow duplicate elements.
 - Supports mathematical set operations.
-

Syntax

```
set_name = {element1, element2, element3}
```

Example

```
numbers = {10, 20, 30, 20}

print(numbers)
```

Output

```
{10, 20, 30}
```

Common Set Methods

1. add()

Adds an element.

```
numbers = {10,20}  
numbers.add(30)  
print(numbers)
```

2. remove()

Removes an element.

```
numbers = {10,20,30}  
numbers.remove(20)  
print(numbers)
```

3. union()

Returns the union of two sets.

```
a = {1,2}  
b = {2,3}  
  
print(a.union(b))
```

Output

```
{1, 2, 3}
```

4. intersection()

Returns common elements.

```
a = {1,2,3}  
b = {2,3,4}  
  
print(a.intersection(b))
```

Output

```
{2, 3}
```

5. difference()

Returns elements present in the first set but not in the second.

```
a = {1, 2, 3}
b = {2, 3, 4}

print(a.difference(b))
```

Output

```
{1}
```

Advantages

- Automatically removes duplicate values.
- Fast searching and membership testing.
- Supports mathematical operations like union, intersection, and difference.
- Easy to use for storing unique data.

UNIT – 5 : OBJECT ORIENTED PROGRAMMING IN PYTHON, FILE HANDLING AND EXCEPTION HANDLING

5.0 Object Oriented Programming (OOP) in Python, File Handling and Exception Handling

Definition

Object-Oriented Programming (OOP) is a programming paradigm based on **classes** and **objects**. It helps organize programs into reusable and modular components.

File Handling is the process of creating, opening, reading, writing, and closing files.

Exception Handling is used to handle runtime errors so that the program does not terminate unexpectedly.

Advantages

- Promotes code reusability.
 - Improves program organization.
 - Simplifies maintenance.
 - Enables efficient error handling.
 - Supports permanent data storage using files.
-

5.1 Creating Classes

Definition

A **class** is a blueprint or template used to create objects. It contains variables (attributes) and functions (methods).

Syntax

```
class ClassName:  
    statements
```

Example

```
class Student:  
    name = "Ravi"  
  
    def display(self):  
        print(self.name)  
  
s = Student()  
s.display()
```

Output

Ravi

Advantages

- Groups related data and functions.
 - Supports code reuse.
 - Improves program structure.
 - Easy to maintain.
-

5.2 Creating Objects

Definition

An **object** is an instance of a class. It is used to access the variables and methods of the class.

Syntax

```
object_name = ClassName()
```

Example

```
class Student:
```

```
def display(self):  
    print("Welcome to Python")  
  
s1 = Student()  
  
s1.display()
```

Output

Welcome to Python

Advantages

- Represents real-world entities.
 - Accesses class members easily.
 - Multiple objects can be created from one class.
-

5.3 Method Overloading and Overriding

A) Method Overloading

Definition

Method overloading means creating methods with the same name but different parameters. Python does **not support true method overloading**, but similar behavior can be achieved using default arguments.

Example

```
class Demo:  
    def add(self, a=0, b=0):  
        print(a + b)  
  
d = Demo()  
  
d.add()  
d.add(10, 20)
```

Output

0
30

Advantages

- Makes methods flexible.
- Reduces the number of method names.
- Simplifies coding.

B) Method Overriding

Definition

Method overriding occurs when a child class provides its own implementation of a method that already exists in the parent class.

Example

```
class Animal:
    def sound(self):
        print("Animal Sound")

class Dog(Animal):
    def sound(self):
        print("Bark")

d = Dog()

d.sound()
```

Output

Bark

Advantages

- Supports runtime polymorphism.
 - Improves flexibility.
 - Allows customized behavior.
-

5.4 Data Hiding

Definition

Data Hiding is the process of restricting direct access to class data. In Python, data hiding is achieved by using **double underscore** (__) before the variable name.

Example

```
class Student:
    __marks = 90

    def display(self):
        print(self.__marks)

s = Student()

s.display()
```

Output

90

Advantages

- Protects data.
 - Prevents unauthorized access.
 - Improves security.
 - Supports encapsulation.
-

5.5 Data Abstraction

Definition

Data Abstraction means hiding implementation details and showing only the essential features of an object.

Python provides abstraction through the **abc (Abstract Base Class)** module.

Example

```
from abc import ABC, abstractmethod

class Shape(ABC):

    @abstractmethod
    def area(self):
        pass
```

Advantages

- Hides unnecessary details.
 - Improves security.
 - Simplifies program design.
 - Makes programs easier to maintain.
-

5.6 Opening Files in Different Modes

Definition

Before reading or writing a file, it must be opened using the `open()` function.

Syntax

```
file = open("filename", "mode")
```

File Modes

Mode	Description
------	-------------

r	Read mode
---	-----------

w	Write mode
---	------------

a	Append mode
---	-------------

x	Create a new file
---	-------------------

Mode	Description
------	-------------

r+	Read and write
----	----------------

w+	Write and read
----	----------------

a+	Append and read
----	-----------------

Example

```
file = open("sample.txt", "r")
```

Advantages

- Supports different file operations.
 - Easy file management.
 - Flexible access to files.
-

5.7 Processing Files

Definition

Processing files means reading data from or writing data to a file.

Writing to a File

Example

```
file = open("sample.txt", "w")  
  
file.write("Welcome to Python")  
  
file.close()
```

Reading from a File

Example

```
file = open("sample.txt", "r")  
  
print(file.read())  
  
file.close()
```

Output

Welcome to Python

Advantages

- Stores data permanently.
 - Easy retrieval of information.
 - Efficient data management.
-

5.8 Closing a File

Definition

After completing file operations, the file should be closed using the `close()` method. Closing a file releases system resources.

Syntax

```
file.close()
```

Example

```
file = open("sample.txt", "r")  
  
print(file.read())  
  
file.close()
```

Advantages

- Frees system resources.
 - Prevents data loss.
 - Ensures proper file handling.
-

5.9 Exception Handling

Definition

Exception Handling is used to handle runtime errors so that the program continues executing without crashing.

Python uses **try**, **except**, **else**, and **finally** blocks.

Syntax

```
try:
    statements
except:
    statements
```

Example 1

```
try:
    a = 10
    b = 0

    print(a / b)

except ZeroDivisionError:
    print("Division by zero is not allowed")
```

Output

Division by zero is not allowed

Example 2

```
try:
    number = int(input("Enter Number: "))
```

```
        print(number)

except ValueError:
    print("Invalid Input")
```

Sample Output

```
Enter Number: abc
Invalid Input
```

Exception Handling Keywords

Keyword	Purpose
try	Contains code that may produce an exception
except	Handles the exception
else	Executes if no exception occurs
finally	Executes whether an exception occurs or not

Advantages

- Prevents program termination.
 - Improves reliability.
 - Makes debugging easier.
 - Handles runtime errors efficiently.
 - Improves user experience.
-